

Java Stream API



Beginner Level – Basic Understanding

1. What is a Stream in Java?
 2. How do Streams differ from Collections?
 3. How do you create a Stream from a list?
 4. What are intermediate and terminal operations in Streams?
 5. What does `.filter()` do?
 6. How is `.map()` different from `.flatMap()`?
 7. How does `.forEach()` work?
 8. What does `.collect()` do? What are collectors?
 9. How to convert a Stream back into a list?
 10. What is the purpose of `.count()` in streams?
 11. How do you sort a Stream?
 12. What's the difference between `.forEach()` and `.map()`?
 13. How can you limit the number of items in a Stream?
 14. What is the use of `.skip()`?
 15. How to remove duplicates from a Stream?
-



Intermediate Level – Functional Concepts

16. How are Streams lazy?
 17. How does short-circuiting work in Streams?
 18. What are common terminal operations in Streams?
 19. How does `.reduce()` work? Give real examples.
 20. What is the difference between `reduce()` and `collect()`?
 21. What are method references in the context of Streams?
 22. How do you convert a Stream to a Map?
 23. What is a Collector? Give examples of custom collectors.
 24. What is the difference between `.findFirst()` and `.findAny()`?
 25. When should you use `.anyMatch()`, `.allMatch()`, and `.noneMatch()`?
 26. What is the purpose of `.peek()`?
 27. What does `.distinct()` do and how does it work internally?
 28. How do you sort a Stream using custom comparator?
 29. What happens if you try to reuse a Stream?
 30. How do you flatten a list of lists using `flatMap()`?
-



Advanced Level – Performance and Internals

31. How does internal iteration differ from external iteration?
32. What are Spliterators and how do they relate to Streams?
33. How are Streams implemented internally in Java?
34. How is memory handled during Stream operations?
35. What is stream fusion or operation chaining?
36. How does short-circuiting affect performance?
37. What happens when exceptions are thrown inside stream operations?
38. What is the cost of boxing/unboxing in streams?

39. How to profile performance in a Stream pipeline?
 40. What is the overhead of using Streams vs. loops?
 41. How does Java optimize the execution of Stream pipelines?
 42. Can you cancel a Stream operation mid-execution?
 43. What is backpressure and does it apply to Java Streams?
 44. Are Streams thread-safe?
 45. When are intermediate operations evaluated?
 46. Are Streams faster than loops? Always?
 47. How do parallel streams partition data internally?
 48. How does garbage collection impact long-living Stream pipelines?
 49. Can a stream work on infinite data?
 50. What is the difference between stateful and stateless operations?
-



Parallel Streams and Concurrency

51. What is a parallel stream?
 52. How do you create one?
 53. How does `.parallelStream()` work internally?
 54. When should you use parallel streams?
 55. What are the downsides of parallel streams?
 56. How does fork-join framework relate to parallel streams?
 57. How are tasks split in parallel streams?
 58. How to ensure thread safety when using parallel streams?
 59. What are side-effects and why are they dangerous in parallel streams?
 60. Can you use `forEachOrdered()` with parallel streams?
 61. How do parallel streams affect performance on different cores?
 62. What's the difference between `.stream().parallel()` and `.parallelStream()`?
-



Real-World Usage and Design Patterns

63. How do you process large CSV or JSON data with streams?
 64. How to use streams with files (`Files.lines()`)?
 65. How to build data pipelines using Stream API?
 66. How to aggregate data using Collectors?
 67. How to implement Group By using Stream?
 68. How to join strings with delimiter using streams?
 69. How to compute average, sum, min, max using streams?
 70. How to generate a list of numbers with `Stream.iterate()`?
 71. How to simulate SQL-like queries using Stream API?
 72. How to use streams in REST APIs?
 73. How to stream large database results?
 74. How to use streams with pagination?
 75. How to convert streams into paged results?
 76. How to combine multiple streams?
 77. How to debug a complex Stream pipeline?
 78. How to write clean and readable Stream code?
-

Testing & Debugging

79. How to test Stream pipelines?
 80. How to write unit tests for methods using streams?
 81. How to debug intermediate Stream stages?
 82. What tools help in debugging streams?
 83. When should you avoid using streams for clarity?
-

Edge Cases and Pitfalls

84. Can a stream be null? How do you handle that?
 85. Why does `stream()` throw `NullPointerException`?
 86. What happens if `.collect()` is called twice?
 87. Why shouldn't you use `forEach()` to modify external variables?
 88. What are side-effects in streams and why are they discouraged?
 89. Why does `.peek()` not always show output?
 90. How to avoid memory leaks in large stream pipelines?
 91. How does exception handling work in streams?
 92. How do you handle checked exceptions in streams?
 93. What are the risks of using `Collectors.toMap()`?
 94. How to avoid key collisions in `Collectors.toMap()`?
 95. What happens if your stream never terminates?
-

Custom Implementations and Extensions

96. How to write a custom collector?
 97. How to create a custom Stream-like class?
 98. How to implement stream processing with backpressure?
 99. How to create a stream from a database cursor?
 100. How to use streams with reactive programming libraries (e.g., RxJava or Reactor)?
 101. How to integrate Java Streams with Java 21's Structured Concurrency?
-

Bonus – Challenge Yourself

- Can you reimplement `Collectors.groupingBy()` from scratch?
- Build your own simple Stream-like class that supports `map`, `filter`, and `collect`.
- Profile performance: Streams vs loops vs parallel streams on 10M records.
- Create a Stream utility that logs each stage for debugging.
- Write a functional, stream-based implementation of a mini SQL engine.